Sesión 7: PROGRAMACIÓN EN EL SERVIDOR Y OBJETOS DE BASE DE DATOS

Vistas, Funciones, Procedimientos, Triggers y su integración con sistemas reales

Por
Carlos César Sánchez Coronel

2026

Índice general

Índice general **3**

1. Introducción **4**

2. Vistas (Views) **5**
2.1. Vistas Simples 5
2.1.1. Casos de Uso de Vistas 5
2.2. Vistas Materializadas 5
2.2.1. Vistas Actualizables 6
2.3. Ejercicio Resuelto: Vista para Informes 6

3. Funciones y Procedimientos Almacenados **7**
3.1. Diferencias entre Funciones y Procedimientos 7
3.2. Funciones Escalares 7
3.3. Funciones Tabla 8
3.4. Procedimientos Almacenados 8
3.5. Manejo de Parámetros y Retorno 10
3.6. Caso de Uso Real: Generación de Tickets en una Gasolinera 10

4. Triggers (Disparadores) **12**
4.1. Tipos de Triggers 12
4.2. Ejemplo: Trigger de Auditoría 12
4.3. Trigger de Validación 13
4.4. Trigger INSTEAD OF en Vistas 13
4.5. Precauciones con Triggers 14

5. Programación Avanzada: Cursores, Excepciones y Transacciones **15**
5.1. Cursores 15
5.2. Manejo de Excepciones 15
5.3. Transacciones dentro de Procedimientos 16

6. Funciones de Hash y Criptografía para Datos Sensibles **17**
6.1. Conceptos Fundamentales 17
6.2. Funciones de Hash en SQL 17
6.2.1. PostgreSQL 17
6.2.2. MySQL 18
6.2.3. SQL Server 18
6.2.4. Oracle 18
6.3. Uso Práctico de Hash 18

6.3.1. Almacenamiento de Contraseñas . . . 18
6.3.2. Anonimización de Datos . . . 18
6.3.3. Integridad de Datos (Checksums) . . . 19
6.4. Cifrado en Base de Datos . . . 19
6.4.1. PostgreSQL con pgcrypto . . . 19
6.4.2. MySQL . . . 19
6.4.3. SQL Server . . . 19
6.5. Consideraciones de Seguridad . . . 19

7. Integración con Sistemas Externos 21
7.1. ¿Por qué integrar la lógica en la base de datos? . . . 21
7.2. Requerimientos Típicos de Integración . . . 21
7.3. Ejecución desde Aplicaciones . . . 21
7.3.1. Python . . . 21
7.3.2. Java (JDBC) . . . 22
7.3.3. C# (ADO.NET) . . . 22
7.3.4. Node.js . . . 23
7.4. Integración con Modelos de Machine Learning / IA . . . 23
7.4.1. Extracción de Datos para Entrenamiento . . . 23
7.4.2. Inferencia en la Base de Datos . . . 24
7.4.3. Caso: Sistema de Recomendación . . . 24
7.5. Integración con ETL y Orquestadores . . . 24
7.5.1. Uso en Apache Airflow . . . 24
7.5.2. Beneficios de encapsular lógica en procedimientos para ETL . . . 25
7.5.3. Ejemplo: Procedimiento de Agregación Diaria . . . 25
7.6. Acceso mediante Credenciales y Gestión de Permisos . . . 25
7.6.1. Creación de Usuarios y Roles . . . 25
7.6.2. SECURITY DEFINER vs INVOKER . . . 26
7.6.3. Buenas Prácticas de Seguridad . . . 26
7.7. Ejecución desde Consola y Dispositivos Móviles . . . 26
7.7.1. Acceso desde Línea de Comandos . . . 26
7.7.2. Aplicaciones Móviles . . . 27
7.8. Integración con Múltiples Motores (Políglota) . . . 27
7.9. Ejercicio Resuelto: Integración Completa . . . 27

Referencias 29

8. Comparativa entre Motores 30

9. Optimización de Código en el Servidor 31
9.0.1. Buenas Prácticas . . . 31
9.0.2. Ejemplo de Optimización . . . 31

10. Ejercicios Resueltos Integradores 33
10.1. Ejercicio 1: Sistema de Ventas con Trigger de Stock . . . 33
10.2. Ejercicio 2: Función para Calcular el Total de un Pedido . . . 34
10.3. Ejercicio 3: Vista de Informes con Datos Sensibles Ocultos . . . 34
10.4. Ejercicio 4: Procedimiento para Transferencia Bancaria con Auditoría . . . 35

11. Glosario de Términos **36**

Referencias **37**

Capítulo 1

Introducción

En las sesiones anteriores hemos aprendido a modelar bases de datos, manipular datos con DML, y realizar análisis avanzados con agregaciones y ventanas. Ahora es el momento de llevar la lógica al servidor: los SGBD modernos permiten programar directamente dentro de la base de datos mediante vistas, funciones, procedimientos almacenados y disparadores (triggers). Esto no solo reduce la latencia de red al ejecutar código cerca de los datos, sino que también centraliza las reglas de negocio, mejora la seguridad y facilita el mantenimiento.

En esta sesión exploraremos en profundidad cada uno de estos objetos, con ejemplos prácticos inspirados en casos reales: generación de tickets en gasolineras, procesos de auditoría en el sector bancario, integración con sistemas externos (como aplicaciones móviles o consolas), y comparativas entre los principales motores (PostgreSQL, MySQL, SQL Server, Oracle). También abordaremos la optimización de código, el manejo de permisos y cómo estos objetos se ejecutan en entornos productivos.

Capítulo 2

Vistas (Views)

Una vista es una tabla virtual basada en el resultado de una consulta. No almacena datos físicamente (excepto las vistas materializadas), sino que presenta los datos de una o más tablas de forma personalizada.

2.1. Vistas Simples

Las vistas simples se crean con una consulta SELECT y se pueden usar como si fueran tablas.

```
CREATE VIEW clientes_activos AS
SELECT id, nombre, email
FROM clientes
WHERE activo = true;
```

Luego se consulta:

```
SELECT * FROM clientes_activos WHERE ciudad = 'Madrid';
```

2.1.1. Casos de Uso de Vistas

- **Seguridad**: Ocultar columnas sensibles (ej. salarios) o filas (ej. solo datos del departamento del usuario).
- **Simplicidad**: Proveer una interfaz simplificada para usuarios de negocio, encapsulando joins complejos.
- **Consistencia**: Garantizar que todos usen la misma lógica de negocio.

2.2. Vistas Materializadas

A diferencia de las vistas normales, las vistas materializadas almacenan físicamente el resultado de la consulta. Se actualizan periódicamente o a petición. Son ideales para reportes que no requieren datos en tiempo real.

```
CREATE MATERIALIZED VIEW resumen_ventas AS
SELECT producto_id, SUM(cantidad) AS total_vendido
FROM ventas
```

```
GROUP BY producto_id;

-- Refrescar la vista (en PostgreSQL)
REFRESH MATERIALIZED VIEW resumen_ventas;
```

En Oracle se usa `REFRESH COMPLETE` o `FAST`, y en SQL Server existen los `indexed views`.

2.2.1. Vistas Actualizables

Algunas vistas simples pueden ser actualizables (permiten INSERT, UPDATE, DELETE) si cumplen ciertas condiciones (una sola tabla, sin funciones de agregación, etc.). En caso contrario, se pueden usar `INSTEAD OF` triggers (lo veremos más adelante).

2.3. Ejercicio Resuelto: Vista para Informes

Enunciado: Crear una vista que muestre para cada cliente su nombre, total gastado y número de pedidos, solo para clientes con al menos un pedido. **Solución:**

```
CREATE VIEW resumen_clientes AS
SELECT c.id, c.nombre, COUNT(p.id) AS num_pedidos, COALESCE(SUM(p
   .total), 0) AS total_gastado
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
GROUP BY c.id, c.nombre
HAVING COUNT(p.id) > 0;
```

Capítulo 3

Funciones y Procedimientos Almacenados

Las funciones y procedimientos permiten encapsular lógica compleja en la base de datos. Se ejecutan en el servidor, reduciendo el tráfico de red y centralizando reglas de negocio.

3.1. Diferencias entre Funciones y Procedimientos

- **Función**: Retorna un valor (escalar o tabla). Se usa dentro de expresiones SQL. No puede realizar transacciones (commit/rollback) en la mayoría de los motores.
- **Procedimiento**: Puede no retornar valor, o retornar múltiples resultados (conjuntos). Puede incluir transacciones. Se invoca con `CALL`.

En la práctica, la sintaxis varía entre motores.

3.2. Funciones Escalares

Ejemplo: Función que calcula el IVA.

```
-- PostgreSQL
CREATE OR REPLACE FUNCTION calcular_iva(monto NUMERIC) RETURNS
   NUMERIC AS $$
BEGIN
    RETURN monto * 0.21;
END;
$$ LANGUAGE plpgsql;

-- Uso
SELECT calcular_iva(100);
```

En SQL Server:

```
CREATE FUNCTION dbo.calcular_iva(@monto NUMERIC(10,2)) RETURNS
   NUMERIC(10,2)
AS
BEGIN
```

```
    RETURN @monto * 0.21;
END;
```

En MySQL:

```
CREATE FUNCTION calcular_iva(monto DECIMAL(10,2)) RETURNS DECIMAL
   (10,2) DETERMINISTIC
BEGIN
    RETURN monto * 0.21;
END;
```

En Oracle:

```
CREATE OR REPLACE FUNCTION calcular_iva(monto NUMBER) RETURN
   NUMBER IS
BEGIN
    RETURN monto * 0.21;
END;
```

3.3. Funciones Tabla

Devuelven un conjunto de filas, como una tabla. Útiles para encapsular consultas complejas.

```
-- PostgreSQL (RETURNS TABLE)
CREATE OR REPLACE FUNCTION empleados_por_depto(depto_id INT)
RETURNS TABLE(id INT, nombre TEXT, salario NUMERIC) AS $$
BEGIN
    RETURN QUERY SELECT id, nombre, salario FROM empleados WHERE
       departamento_id = depto_id;
END;
$$ LANGUAGE plpgsql;

-- Uso
SELECT * FROM empleados_por_depto(2);
```

En SQL Server son `TVF` (Table-Valued Functions):

```
CREATE FUNCTION dbo.empleados_por_depto(@depto_id INT) RETURNS
   TABLE
AS
RETURN (SELECT id, nombre, salario FROM empleados WHERE
   departamento_id = @depto_id);
```

3.4. Procedimientos Almacenados

Los procedimientos realizan acciones (inserciones, actualizaciones) y pueden tener parámetros de entrada y salida.

```
-- PostgreSQL (procedimiento, requiere CREATE PROCEDURE desde
   PG11)
```

```
CREATE OR REPLACE PROCEDURE transferir(origen INT, destino INT,
   monto NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE cuentas SET saldo = saldo - monto WHERE id = origen;
    UPDATE cuentas SET saldo = saldo + monto WHERE id = destino;
    COMMIT; -- o confiar en que la transacci n externa haga
       commit
END;
$$;

-- Llamada
CALL transferir(1, 2, 100);
```

En SQL Server:

```
CREATE PROCEDURE transferir @origen INT, @destino INT, @monto
   NUMERIC(10,2)
AS
BEGIN
    BEGIN TRANSACTION;
    UPDATE cuentas SET saldo = saldo - @monto WHERE id = @origen;
    UPDATE cuentas SET saldo = saldo + @monto WHERE id = @destino
       ;
    COMMIT;
END;
```

En MySQL:

```
DELIMITER //
CREATE PROCEDURE transferir(IN origen INT, IN destino INT, IN
   monto DECIMAL(10,2))
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION ROLLBACK;
    START TRANSACTION;
    UPDATE cuentas SET saldo = saldo - monto WHERE id = origen;
    UPDATE cuentas SET saldo = saldo + monto WHERE id = destino;
    COMMIT;
END //
DELIMITER ;
```

En Oracle:

```
CREATE OR REPLACE PROCEDURE transferir(origen NUMBER, destino
   NUMBER, monto NUMBER) IS
BEGIN
    UPDATE cuentas SET saldo = saldo - monto WHERE id = origen;
    UPDATE cuentas SET saldo = saldo + monto WHERE id = destino;
    COMMIT;
END;
```

3.5. Manejo de Parámetros y Retorno

Los procedimientos pueden tener parámetros de salida. Ejemplo en SQL Server:

```
CREATE PROCEDURE obtener_nombre_cliente @id INT, @nombre VARCHAR
   (100) OUTPUT
AS
BEGIN
    SELECT @nombre = nombre FROM clientes WHERE id = @id;
END;

DECLARE @nom VARCHAR(100);
EXEC obtener_nombre_cliente 1, @nom OUTPUT;
PRINT @nom;
```

3.6. Caso de Uso Real: Generación de Tickets en una Gasolinera

Imaginemos un sistema de gasolineras donde cada vez que un cliente carga combustible, se genera un ticket con número correlativo por estación y fecha. Podemos usar una función o procedimiento que:

1. Obtenga el último número de ticket para esa estación y fecha.
2. Inserte la venta con el nuevo número.
3. Retorne el ticket generado.

Ejemplo en PostgreSQL:

```
CREATE TABLE ventas_gasolina (
    id SERIAL PRIMARY KEY,
    estacion_id INT NOT NULL,
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
    ticket_num INT NOT NULL,
    importe NUMERIC NOT NULL,
    UNIQUE (estacion_id, fecha, ticket_num)
);

CREATE OR REPLACE FUNCTION generar_ticket(p_estacion INT,
   p_importe NUMERIC)
RETURNS INT AS $$
DECLARE
    v_ticket INT;
BEGIN
    -- Obtener el m ximo ticket del d a para esa estaci n
    SELECT COALESCE(MAX(ticket_num), 0) + 1 INTO v_ticket
    FROM ventas_gasolina
    WHERE estacion_id = p_estacion AND fecha = CURRENT_DATE;

    -- Insertar la venta
```

```
    INSERT INTO ventas_gasolina (estacion_id, fecha, ticket_num,
       importe)
    VALUES (p_estacion, CURRENT_DATE, v_ticket, p_importe);

    RETURN v_ticket;
END;
$$ LANGUAGE plpgsql;

-- Llamada desde la aplicaci n
SELECT generar_ticket(5, 45.50);
```

Este procedimiento garantiza la integridad del correlativo incluso con concurrencia (gracias al lock implícito de la tabla).

Capítulo 4

Triggers (Disparadores)

Un trigger es un bloque de código que se ejecuta automáticamente ante un evento (INSERT, UPDATE, DELETE) sobre una tabla. Puede ser BEFORE, AFTER o INSTEAD OF, y puede ejecutarse por fila o por sentencia.

4.1. Tipos de Triggers

- **BEFORE**: Se ejecuta antes de la operación. Útil para validaciones o modificar valores antes de insertar.
- **AFTER**: Después de la operación. Ideal para auditoría, sincronización.
- **INSTEAD OF**: Reemplaza la operación. Se usa en vistas para hacerlas actualizables.

4.2. Ejemplo: Trigger de Auditoría

Supongamos que queremos auditar los cambios de salario en una tabla empleados.

```
-- Tabla de auditor a
CREATE TABLE audit_salarios (
    id_empleado INT,
    salario_anterior NUMERIC,
    salario_nuevo NUMERIC,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    usuario TEXT DEFAULT current_user
);

-- Trigger en PostgreSQL
CREATE OR REPLACE FUNCTION audit_salario_func()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.salario IS DISTINCT FROM NEW.salario THEN
        INSERT INTO audit_salarios (id_empleado, salario_anterior
            , salario_nuevo)
        VALUES (OLD.id, OLD.salario, NEW.salario);
    END IF;
    RETURN NEW;
```

```
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_audit_salario
AFTER UPDATE OF salario ON empleados
FOR EACH ROW EXECUTE FUNCTION audit_salario_func();
```

4.3. Trigger de Validación

Evitar que se inserten empleados con salario negativo.

```
CREATE OR REPLACE FUNCTION validar_salario()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.salario < 0 THEN
        RAISE EXCEPTION 'El salario no puede ser negativo';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_validar_salario
BEFORE INSERT OR UPDATE ON empleados
FOR EACH ROW EXECUTE FUNCTION validar_salario();
```

4.4. Trigger INSTEAD OF en Vistas

Si tenemos una vista compleja que queremos actualizar, podemos definir un trigger INSTEAD OF.

```
-- Vista que combina dos tablas
CREATE VIEW empleados_departamentos AS
SELECT e.id, e.nombre, e.salario, d.nombre AS depto
FROM empleados e
JOIN departamentos d ON e.departamento_id = d.id;

-- Trigger para permitir INSERT en la vista (solo en PostgreSQL
   se puede con reglas, o en Oracle)
-- En PostgreSQL, las vistas actualizables requieren reglas o
   triggers INSTEAD OF (desde versi n 9.1 con BEFORE triggers? En
    realidad, PostgreSQL no tiene INSTEAD OF triggers para vistas,
    usa reglas. En SQL Server s  .
-- Para simplificar, mostramos en SQL Server:
CREATE TRIGGER tr_vista_empleados
ON empleados_departamentos
INSTEAD OF INSERT
AS
BEGIN
    INSERT INTO empleados (nombre, salario, departamento_id)
```

```
    SELECT i.nombre, i.salario, d.id
    FROM inserted i
    JOIN departamentos d ON i.depto = d.nombre;
END;
```

En PostgreSQL se usarían reglas (RULE), pero es más complejo.

4.5. Precauciones con Triggers

- Pueden causar recursividad (trigger que actualiza la misma tabla y se dispara a sí mismo). Se debe controlar con condiciones.
- Afectan el rendimiento, especialmente si son por fila en operaciones masivas.
- Dificultan la depuración porque la lógica está oculta.

Capítulo 5

Programación Avanzada: Cursores, Excepciones y Transacciones

En lenguajes procedurales como PL/pgSQL, T-SQL o PL/SQL, podemos usar estructuras de control complejas.

5.1. Cursores

Un cursor permite recorrer filas una por una. Aunque es más ineficiente que las operaciones basadas en conjuntos, a veces es necesario.

```
-- PostgreSQL: ejemplo de cursor
CREATE OR REPLACE FUNCTION procesar_empleados()
RETURNS VOID AS $$
DECLARE
    cur CURSOR FOR SELECT id, salario FROM empleados WHERE activo
        = true;
    v_id INT;
    v_salario NUMERIC;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_id, v_salario;
        EXIT WHEN NOT FOUND;
        -- Procesar cada empleado
        UPDATE empleados SET salario = v_salario * 1.05 WHERE id
           = v_id;
    END LOOP;
    CLOSE cur;
END;
$$ LANGUAGE plpgsql;
```

En SQL Server, los cursores son similares pero con sintaxis propia.

5.2. Manejo de Excepciones

Capturar errores para evitar que todo falle.

```
-- PostgreSQL
CREATE OR REPLACE FUNCTION dividir(a NUMERIC, b NUMERIC) RETURNS
   NUMERIC AS $$
BEGIN
    RETURN a / b;
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE 'Divisi n por cero';
        RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

5.3. Transacciones dentro de Procedimientos

En procedimientos, podemos controlar transacciones manualmente. En PostgreSQL, los procedimientos permiten COMMIT/ROLLBACK, mientras que las funciones no.

Capítulo 6

Funciones de Hash y Criptografía para Datos Sensibles

En entornos productivos, especialmente en aplicaciones financieras, de salud o que manejan datos personales, la protección de la información es crítica. Las bases de datos ofrecen funciones para hashear y cifrar datos, garantizando confidencialidad e integridad.

6.1. Conceptos Fundamentales

- **Hash**: Función unidireccional que transforma una entrada en una cadena de longitud fija (resumen). No es reversible. Ejemplos: MD5, SHA-1, SHA-2 (SHA-256, SHA-512).
- **Cifrado**: Proceso reversible que transforma datos mediante una clave. Puede ser simétrico (misma clave para cifrar/descifrar) o asimétrico (clave pública/privada).
- **Sal (Salt)**: Valor aleatorio añadido al hash para evitar ataques de diccionario y rainbow tables.

6.2. Funciones de Hash en SQL

Los principales motores incluyen funciones hash. Es importante conocerlas para tareas de anonimización, integridad y almacenamiento seguro de contraseñas.

6.2.1. PostgreSQL

```
-- MD5 (r pido pero inseguro para contrase as)
SELECT md5('mi_secreto'); -- Devuelve hash hexadecimal

-- SHA-256 (disponible en el m dulo pgcrypto)
CREATE EXTENSION IF NOT EXISTS pgcrypto;
SELECT encode(digest('mi_secreto', 'sha256'), 'hex');
```

6.2.2. MySQL

```
SELECT MD5('mi_secreto');
SELECT SHA2('mi_secreto', 256); -- SHA-256
```

6.2.3. SQL Server

SELECT HASHBYTES('MD5', 'mi$_secreto'); $SELECTHASHBYTES('SHA2_256', 'mi_secreto');$

6.2.4. Oracle

```
SELECT DBMS_CRYPTO.HASH(UTL_RAW.CAST_TO_RAW('mi_secreto'),
   DBMS_CRYPTO.HASH_SH256) FROM DUAL;
```

6.3. Uso Práctico de Hash

6.3.1. Almacenamiento de Contraseñas

Nunca se deben guardar contraseñas en texto plano. Se almacena el hash con salt.

```
-- Tabla de usuarios
CREATE TABLE usuarios (
    id SERIAL PRIMARY KEY,
    email TEXT UNIQUE,
    password_hash TEXT,
    salt TEXT
);

-- Ejemplo de inserci n con salt aleatorio
INSERT INTO usuarios (email, password_hash, salt)
VALUES ('usuario@mail.com',
        encode(digest('password123' || 's1', 'sha256'), 'hex'),
        's1');
```

Para verificar, se calcula el hash con el mismo salt y se compara.

6.3.2. Anonimización de Datos

En entornos de desarrollo o testing, se pueden hashear datos personales para cumplir normativas (GDPR).

```
-- Sustituir email real por un hash irreversible
UPDATE clientes SET email = md5(email) WHERE entorno = 'test';
```

Nota: Esto no es anonimización fuerte si el dominio es pequeño (ataques de diccionario). Se recomienda usar SHA-256 con salt.

6.3.3. Integridad de Datos (Checksums)

Se puede almacenar el hash de una fila para detectar modificaciones no autorizadas.

```
ALTER TABLE productos ADD COLUMN checksum TEXT;

-- Calcular checksum al insertar
CREATE OR REPLACE FUNCTION calcular_checksum() RETURNS TRIGGER AS
    $$
BEGIN
    NEW.checksum = encode(digest(NEW.nombre || NEW.precio::text,
        'sha256'), 'hex');
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

6.4. Cifrado en Base de Datos

Algunos motores permiten cifrado de columnas o datos. Es útil para información extremadamente sensible (números de tarjeta, datos médicos).

6.4.1. PostgreSQL con pgcrypto

```
-- Cifrado sim trico (clave secreta)
SELECT encrypt('dato_sensible', 'clave', 'aes');

-- Descifrar
SELECT decrypt(dato_cifrado, 'clave', 'aes');
```

Requiere gestión segura de la clave (no hardcodear en código).

6.4.2. MySQL

Funciones AES_ENCRYPT / AES_DECRYPT.

```
SELECT AES_ENCRYPT('dato', 'clave');
SELECT AES_DECRYPT(dato_cifrado, 'clave');
```

6.4.3. SQL Server

Funciones de cifrado de nivel de columna (Always Encrypted) o funciones como ENCRYPTBYPASSPHRASE.

6.5. Consideraciones de Seguridad

- Las funciones hash son rápidas pero para contraseñas se recomienda usar algoritmos específicos como bcrypt, scrypt o PBKDF2 (a nivel de aplicación, no en BD pura).

- Las claves de cifrado deben almacenarse fuera de la base de datos (en un vault, variable de entorno).
- El cifrado en BD puede afectar el rendimiento y el uso de índices.
- En entrevistas, suelen preguntar cómo almacenarías contraseñas de forma segura (hash + salt) y cómo manejarías datos sensibles.

Capítulo 7

Integración con Sistemas Externos

Los objetos de base de datos (vistas, funciones, procedimientos) no existen en el vacío; son la columna vertebral de aplicaciones, pipelines de datos y sistemas de inteligencia artificial. Entender cómo se integran con el mundo exterior es fundamental para arquitectos de software y ingenieros de datos.

7.1. ¿Por qué integrar la lógica en la base de datos?

- **Latencia**: Ejecutar código cerca de los datos evita transferir grandes volúmenes a la aplicación.
- **Consistencia**: Las reglas de negocio se centralizan y son aplicadas por todas las aplicaciones.
- **Seguridad**: Se puede controlar el acceso a nivel de fila/columna y auditar.
- **Rendimiento**: Las operaciones complejas (joins, agregaciones) son más rápidas en el motor.

7.2. Requerimientos Típicos de Integración

- **Concurrencia**: Múltiples usuarios/aplicaciones accediendo simultáneamente.
- **Latencia**: Respuestas en milisegundos para aplicaciones interactivas.
- **Seguridad**: Autenticación, autorización, cifrado en tránsito (TLS).
- **Disponibilidad**: La base de datos debe estar siempre accesible (alta disponibilidad).
- **Escalabilidad**: Capacidad de manejar crecimiento de datos y peticiones.

7.3. Ejecución desde Aplicaciones

7.3.1. Python

Python es el lenguaje más usado en ciencia de datos y desarrollo backend. Las bibliotecas principales son:

- **psycopg2** para PostgreSQL.
- **pymysql** / **mysql-connector** para MySQL.
- **pyodbc** para SQL Server y otros.
- **cx_Oracle** para Oracle.

Ejemplo con psycopg2 llamando a un procedimiento:

```
import psycopg2
conn = psycopg2.connect("dbname=test user=postgres")
cur = conn.cursor()
# Llamar a funci n que retorna un valor
cur.callproc('generar_ticket', [5, 45.50])
ticket = cur.fetchone()[0]
print("Ticket generado:", ticket)
# Llamar a procedimiento (sin retorno)
cur.callproc('transferir', [1, 2, 100])
conn.commit()
cur.close()
conn.close()
```

Para SQLAlchemy (ORM):

```
from sqlalchemy import create_engine, text
engine = create_engine('postgresql://user:pass@localhost/test')
with engine.connect() as conn:
    result = conn.execute(text("SELECT generar_ticket(:est, :imp)
       "), {"est": 5, "imp": 45.50})
    ticket = result.scalar()
```

7.3.2. Java (JDBC)

```
import java.sql.*;
Connection conn = DriverManager.getConnection("jdbc:postgresql://
   localhost/test", "user", "pass");
CallableStatement stmt = conn.prepareCall("{ ? = call
   generar_ticket(?, ?) }");
stmt.registerOutParameter(1, Types.INTEGER);
stmt.setInt(2, 5);
stmt.setDouble(3, 45.50);
stmt.execute();
int ticket = stmt.getInt(1);
```

7.3.3. C# (ADO.NET)

```
using (var conn = new NpgsqlConnection("Host=localhost;Database=
   test;Username=user;Password=pass"))
{
    conn.Open();
```

```
    using (var cmd = new NpgsqlCommand("SELECT generar_ticket(
        @est, @imp)", conn))
    {
        cmd.Parameters.AddWithValue("est", 5);
        cmd.Parameters.AddWithValue("imp", 45.50);
        var ticket = (int)cmd.ExecuteScalar();
    }
}
```

7.3.4. Node.js

Con librerías como pg para PostgreSQL:

```
const { Client } = require('pg');
const client = new Client({ database: 'test', user: 'postgres' })
    ;
await client.connect();
const res = await client.query('SELECT generar_ticket($1, $2) as
    ticket', [5, 45.50]);
console.log(res.rows[0].ticket);
await client.end();
```

7.4. Integración con Modelos de Machine Learning / IA

Los modelos de IA necesitan datos para entrenamiento e inferencia. La base de datos puede proveerlos mediante funciones o vistas.

7.4.1. Extracción de Datos para Entrenamiento

Se puede crear una función que devuelva un conjunto de datos listo para entrenar (features + etiquetas).

```
CREATE OR REPLACE FUNCTION datos_entrenamiento()
RETURNS TABLE(edad INT, salario NUMERIC, compras INT, target
    BOOLEAN) AS $$
BEGIN
    RETURN QUERY SELECT edad, salario, compras, default_credito
        FROM clientes;
END;
$$ LANGUAGE plpgsql;
```

En Python:

```
import pandas as pd
import psycopg2
conn = psycopg2.connect("...")
df = pd.read_sql("SELECT * FROM datos_entrenamiento()", conn)
# Entrenar modelo...
```

7.4.2. Inferencia en la Base de Datos

Algunos motores permiten ejecutar modelos directamente (PostgreSQL con PL/Python, o mediante extensiones como `postgresml`). Por ejemplo, con PL/Python:

```
CREATE EXTENSION plpython3u;
CREATE OR REPLACE FUNCTION predecir_credito(edad INT, salario
   NUMERIC)
RETURNS BOOLEAN AS $$
    import pickle
    model = pickle.load(open('/ruta/modelo.pkl', 'rb'))
    return model.predict([[edad, salario]])[0]
$$ LANGUAGE plpython3u;
```

Esto evita mover datos, pero requiere cuidado con la seguridad y la gestión de modelos.

7.4.3. Caso: Sistema de Recomendación

Un procedimiento puede recibir un ID de usuario y devolver productos recomendados consultando una tabla de embeddings o usando una función de similitud (pgvector).

```
CREATE OR REPLACE FUNCTION recomendar_productos(usuario_id INT)
RETURNS TABLE(producto_id INT, score FLOAT) AS $$
BEGIN
    RETURN QUERY
    SELECT p.id, (p.embedding <=> (SELECT embedding FROM usuarios
        WHERE id = usuario_id)) AS sim
    FROM productos p
    ORDER BY sim DESC
    LIMIT 10;
END;
$$ LANGUAGE plpgsql;
```

7.5. Integración con ETL y Orquestadores

Los procedimientos almacenados son ideales para tareas pesadas de transformación dentro de un pipeline ETL.

7.5.1. Uso en Apache Airflow

Airflow puede ejecutar SQL mediante hooks específicos. Ejemplo de un DAG que llama a un procedimiento:

```
from airflow import DAG
from airflow.providers.postgres.operators.postgres import
   PostgresOperator
from datetime import datetime

with DAG('etl_diario', start_date=datetime(2025,1,1), schedule='
   @daily') as dag:
    t1 = PostgresOperator(
```

```
        task_id='actualizar_resumen',
        postgres_conn_id='mi_conexion',
        sql="CALL actualizar_resumen_ventas();"
    )
```

También se puede usar `SQLExecuteQueryOperator` en versiones recientes.

7.5.2. Beneficios de encapsular lógica en procedimientos para ETL

- **Rendimiento**: Las transformaciones se ejecutan cerca de los datos.
- **Mantenibilidad**: La lógica está en un solo lugar (la BD) y puede ser reutilizada por diferentes pipelines.
- **Atomicidad**: Se pueden manejar transacciones complejas dentro del procedimiento.

7.5.3. Ejemplo: Procedimiento de Agregación Diaria

```sql
CREATE OR REPLACE PROCEDURE actualizar_resumen_ventas()
LANGUAGE plpgsql
AS $$
BEGIN
    DELETE FROM resumen_diario WHERE fecha = CURRENT_DATE - 1;
    INSERT INTO resumen_diario (fecha, producto_id, total_vendido
       )
    SELECT CURRENT_DATE - 1, producto_id, SUM(cantidad)
    FROM ventas
    WHERE fecha = CURRENT_DATE - 1
    GROUP BY producto_id;
END;
$$;
```

Este procedimiento puede ser invocado desde Airflow cada mañana.

7.6. Acceso mediante Credenciales y Gestión de Permisos

Para que las aplicaciones puedan ejecutar objetos, es necesario otorgar permisos adecuados y gestionar las credenciales de forma segura.

7.6.1. Creación de Usuarios y Roles

```sql
-- PostgreSQL
CREATE USER app_user WITH PASSWORD 'secure_password';
GRANT CONNECT ON DATABASE mi_db TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
GRANT EXECUTE ON FUNCTION generar_ticket(INT, NUMERIC) TO
   app_user;
```

7.6.2. SECURITY DEFINER vs INVOKER

- **SECURITY INVOKER** (por defecto): La función se ejecuta con los privilegios del usuario que la llama.
- **SECURITY DEFINER**: Se ejecuta con los privilegios del creador. Útil para tareas que requieren permisos elevados (por ejemplo, insertar en tablas de auditoría sin dar permisos directos).

Ejemplo en PostgreSQL:

```
CREATE OR REPLACE FUNCTION eliminar_usuario(uid INT)
RETURNS VOID
SECURITY DEFINER
AS $$
BEGIN
    DELETE FROM usuarios WHERE id = uid;
END;
$$ LANGUAGE plpgsql;

REVOKE ALL ON usuarios FROM app_user;
GRANT EXECUTE ON FUNCTION eliminar_usuario(INT) TO app_user;
-- app_user puede eliminar usuarios sin tener DELETE directo en
   la tabla
```

7.6.3. Buenas Prácticas de Seguridad

- No hardcodear credenciales en el código de la aplicación; usar variables de entorno o secretos.
- Usar conexiones TLS (SSL) para evitar sniffing.
- Limitar los privilegios al mínimo necesario (principio de mínimo privilegio).
- Rotar contraseñas periódicamente.
- Para entornos cloud, usar IAM y secretos gestionados (AWS Secrets Manager, Azure Key Vault).

7.7. Ejecución desde Consola y Dispositivos Móviles

7.7.1. Acceso desde Línea de Comandos

Herramientas como `psql`, `mysql`, `sqlcmd` permiten ejecutar procedimientos directamente:

```
psql -U app_user -d mi_db -c "SELECT generar_ticket(5, 45.50);"
```

Esto es útil para pruebas o scripts batch.

7.7.2. Aplicaciones Móviles

No es recomendable conectar una app móvil directamente a la base de datos (riesgos de seguridad, exposición de credenciales). En su lugar, se debe exponer una API REST/GraphQL (por ejemplo, con PostgREST, Hasura, o un backend en Node/Java) que internamente invoque los procedimientos. La API gestiona autenticación y autorización.

7.8. Integración con Múltiples Motores (Políglota)

En arquitecturas complejas, diferentes motores coexisten. Un orquestador como Airflow puede unificarlos.

- Un procedimiento en PostgreSQL puede poblar una tabla, y luego Airflow llama a un procedimiento en MySQL para replicar datos.
- Las funciones de hash y cifrado pueden usarse de manera consistente entre motores.
- Herramientas como Apache Kafka (CDC) pueden capturar cambios en una BD y propagarlos a otra.

7.9. Ejercicio Resuelto: Integración Completa

Enunciado: Diseñar un flujo donde una aplicación web registra pedidos. Cada pedido dispara un trigger que actualiza el stock y genera un registro en una tabla de auditoría. Un proceso ETL diario (Airflow) ejecuta un procedimiento que calcula las ventas por producto y las almacena en una tabla de resumen. Además, un modelo de ML en Python necesita acceder a los datos de ventas para predecir la demanda.

Solución:

1. **Base de datos**: Tablas `productos`, `pedidos`, `detalle_pedido`, `auditoria_stock`, `resumen_ventas`.
2. **Trigger** para actualizar stock y auditar (similar a ejercicios anteriores).
3. **Procedimiento de resumen**:

```
CREATE OR REPLACE PROCEDURE calcular_resumen_diario()
LANGUAGE plpgsql AS $$
BEGIN
    DELETE FROM resumen_ventas WHERE fecha = CURRENT_DATE -
       1;
    INSERT INTO resumen_ventas (fecha, producto_id, total)
    SELECT CURRENT_DATE - 1, dp.producto_id, SUM(dp.cantidad
       * p.precio)
    FROM detalle_pedido dp
    JOIN pedidos pe ON dp.pedido_id = pe.id
    JOIN productos p ON dp.producto_id = p.id
    WHERE pe.fecha::date = CURRENT_DATE - 1
    GROUP BY dp.producto_id;
END;
$$;
```

4. **Airflow DAG** (diario):

```
from airflow import DAG
from airflow.providers.postgres.operators.postgres import
   PostgresOperator
from datetime import datetime

with DAG('ventas_daily', start_date=datetime(2025,1,1),
   schedule='@daily') as dag:
    resumen = PostgresOperator(
        task_id='calcular_resumen',
        postgres_conn_id='postgres_default',
        sql="CALL calcular_resumen_diario();"
    )
    entrenar_modelo = PythonOperator(
        task_id='entrenar_modelo',
        python_callable=entrenar_modelo_func
    )
    resumen >> entrenar_modelo
```

5. **Función para ML**:

```
CREATE OR REPLACE FUNCTION datos_entrenamiento(fecha_inicio
   DATE, fecha_fin DATE)
RETURNS TABLE(fecha DATE, producto_id INT, total NUMERIC) AS
   $$
BEGIN
    RETURN QUERY SELECT fecha, producto_id, total FROM
       resumen_ventas
    WHERE fecha BETWEEN fecha_inicio AND fecha_fin;
END;
$$ LANGUAGE plpgsql;
```

6. **Python para entrenamiento**:

```
import pandas as pd
from sklearn.linear_model import LinearRegression
conn = psycopg2.connect("...")
df = pd.read_sql("SELECT * FROM datos_entrenamiento
   ('2025-01-01', '2025-03-01')", conn)
# Procesar y entrenar...
```

Referencias Adicionales

- OWASP Guía de Almacenamiento de Contraseñas: `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`
- Documentación de pgcrypto: `https://www.postgresql.org/docs/current/pgcrypto.html`
- Apache Airflow: `https://airflow.apache.org/`
- "Designing Data-Intensive Applications"de Martin Kleppmann (capítulos sobre integración y arquitectura).

Capítulo 8

Comparativa entre Motores

A continuación, una tabla resumen de las características de programación en los principales SGBD:

Característica	**PostgreSQL**	**MySQL**
Lenguaje procedural	PL/pgSQL, Python, Perl, etc.	SQL/PSM
Funciones escalares	Sí	Sí
Funciones tabla	Sí (RETURNS TABLE)	Sí (RETU
Procedimientos	Sí (CREATE PROCEDURE)	Sí
Triggers	BEFORE/AFTER/INSTEAD OF (vistas con reglas)	BEFORE/
Cursores	Sí	Sí
Manejo de excepciones	EXCEPTION	DECLARE
Transacciones en procedimientos	Solo en PROCEDURE (no en FUNCTION)	Sí
Vistas materializadas	Sí (REFRESH)	No (simula
Seguridad definer	SECURITY DEFINER	SQL SECU
Soporte de JSON	Sí (JSONB)	Sí

Capítulo 9

Optimización de Código en el Servidor

Escribir código eficiente es crucial para el rendimiento.

9.0.1. Buenas Prácticas

- **Usar operaciones basadas en conjuntos** en lugar de cursores siempre que sea posible. Los cursores son lentos.
- **Evitar triggers innecesarios**: cada trigger añade overhead.
- **Indexar las columnas usadas en condiciones** dentro de las funciones.
- **Usar EXPLAIN** para analizar el rendimiento de las consultas dentro de funciones.
- **Marcar funciones como IMMUTABLE, STABLE o VOLATILE** (en PostgreSQL) para ayudar al optimizador.
- **Evitar la recursión excesiva** en triggers.

9.0.2. Ejemplo de Optimización

Supongamos una función que suma salarios por departamento. Una mala implementación usaría un cursor; una buena usaría SQL directo.

```
-- Ineficiente con cursor
CREATE OR REPLACE FUNCTION suma_salarios_depto(p_depto INT)
   RETURNS NUMERIC AS $$
DECLARE
    total NUMERIC := 0;
    r RECORD;
BEGIN
    FOR r IN SELECT salario FROM empleados WHERE departamento_id
       = p_depto LOOP
        total := total + r.salario;
    END LOOP;
    RETURN total;
END; $$ LANGUAGE plpgsql;
```

```
-- Eficiente
CREATE OR REPLACE FUNCTION suma_salarios_depto(p_depto INT)
   RETURNS NUMERIC AS $$
BEGIN
    RETURN (SELECT SUM(salario) FROM empleados WHERE
       departamento_id = p_depto);
END; $$ LANGUAGE plpgsql;
```

Capítulo 10

Ejercicios Resueltos Integradores

10.1. Ejercicio 1: Sistema de Ventas con Trigger de Stock

Enunciado: En una tienda, cada vez que se inserta una línea de pedido (detalle_pedido), se debe actualizar el stock del producto restando la cantidad vendida. Si el stock queda negativo, se debe abortar la operación. Implementar un trigger en PostgreSQL.

Solución:

```
CREATE OR REPLACE FUNCTION actualizar_stock()
RETURNS TRIGGER AS $$
DECLARE
    v_stock_actual INT;
BEGIN
    -- Obtener stock actual
    SELECT stock INTO v_stock_actual FROM productos WHERE id =
       NEW.producto_id;

    IF v_stock_actual < NEW.cantidad THEN
        RAISE EXCEPTION 'Stock insuficiente para producto %', NEW
           .producto_id;
    END IF;

    UPDATE productos SET stock = stock - NEW.cantidad WHERE id =
       NEW.producto_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_actualizar_stock
BEFORE INSERT ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION actualizar_stock();
```

10.2. Ejercicio 2: Función para Calcular el Total de un Pedido

Enunciado: Crear una función que, dado un pedido_id, calcule el total sumando (cantidad * precio) de sus líneas y actualice la columna total en la tabla pedidos.

Solución:

```
CREATE OR REPLACE FUNCTION calcular_total_pedido(p_pedido_id INT)
RETURNS NUMERIC AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT SUM(dp.cantidad * p.precio) INTO v_total
    FROM detalle_pedido dp
    JOIN productos p ON dp.producto_id = p.id
    WHERE dp.pedido_id = p_pedido_id;

    UPDATE pedidos SET total = v_total WHERE id = p_pedido_id;
    RETURN v_total;
END;
$$ LANGUAGE plpgsql;

-- Llamada
SELECT calcular_total_pedido(10);
```

10.3. Ejercicio 3: Vista de Informes con Datos Sensibles Ocultos

Enunciado: Crear una vista para el departamento de recursos humanos que muestre nombre, salario y departamento, pero oculte el salario a los usuarios que no pertenezcan a RH. (Usar seguridad a nivel de vista con CURRENT$_USER$)

Solución:

```
CREATE VIEW empleados_rh AS
SELECT id, nombre,
       CASE WHEN CURRENT_USER = 'rh_user' THEN salario ELSE NULL
          END AS salario,
       departamento_id
FROM empleados;

-- Luego otorgar permisos SELECT a rh_user y a otros, pero el
   salario ser  NULL para otros.
```

Esto es una forma básica; también se puede combinar con políticas de seguridad (RLS en PostgreSQL).

10.4. Ejercicio 4: Procedimiento para Transferencia Bancaria con Auditoría

Enunciado: Crear un procedimiento que realice una transferencia entre cuentas y registre la operación en una tabla de auditoría.

Solución:

```
CREATE TABLE auditoria_transferencias (
    id SERIAL PRIMARY KEY,
    origen INT,
    destino INT,
    monto NUMERIC,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    usuario TEXT DEFAULT current_user
);

CREATE OR REPLACE PROCEDURE transferir_con_auditoria(origen INT,
   destino INT, monto NUMERIC)
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE cuentas SET saldo = saldo - monto WHERE id = origen;
    UPDATE cuentas SET saldo = saldo + monto WHERE id = destino;

    INSERT INTO auditoria_transferencias (origen, destino, monto)
        VALUES (origen, destino, monto);
    COMMIT;
END;
$$;
```

Capítulo 11

Glosario de Términos

Vista Tabla virtual basada en una consulta.

Vista materializada Vista que almacena físicamente los datos.

Función escalar Función que retorna un valor simple.

Función tabla Función que retorna un conjunto de filas.

Procedimiento almacenado Bloque de código que realiza acciones.

Trigger Código que se ejecuta automáticamente ante un evento.

BEFORE trigger Se ejecuta antes de la operación.

AFTER trigger Se ejecuta después de la operación.

INSTEAD OF trigger Reemplaza la operación (en vistas).

PL/pgSQL Lenguaje procedural de PostgreSQL.

T-SQL Lenguaje procedural de SQL Server.

PL/SQL Lenguaje procedural de Oracle.

SQL/PSM Estándar SQL para procedural, usado en MySQL.

Cursores Mecanismo para recorrer filas una a una.

Seguridad definer El código se ejecuta con permisos del creador.

Seguridad invoker Se ejecuta con permisos del que llama.

Referencias

- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems*. 7th ed. Pearson.
- Documentación oficial de PostgreSQL: `https://www.postgresql.org/docs/`
- Documentación de MySQL: `https://dev.mysql.com/doc/`
- Documentación de SQL Server: `https://docs.microsoft.com/en-us/sql/`
- Documentación de Oracle: `https://docs.oracle.com/en/database/`
- "SQL Performance Explained"de Markus Winand.